

5.11 一些限制因素

我们已经看到在一个程序的数据流图表示中，关键路径指明了执行该程序所需时间的一个基本的下界。也就是说，如果程序中有某条数据相关链，这条链上的所有延迟之和等于 T ，那么这个程序至少需要 T 个周期才能执行完。

我们还看到功能单元的吞吐量界限也是程序执行时间的一个下界。也就是说，假设一个程序一共需要 N 个某种运算的计算，而微处理器只有 C 个能执行这个操作的功能单元，并且这些单元的发射时间为 I 。那么，这个程序的执行至少需要 $N \cdot I/C$ 个周期。

在本节中，我们会考虑其他一些制约程序在实际机器上性能的因素。

5.11.1 寄存器溢出

循环并行性的好处受汇编代码描述计算的能力限制。如果我们的并行度 p 超过了可用的寄存器数量，那么编译器会诉诸溢出(spilling)，将某些临时值存放到内存中，通常是在运行时堆栈上分配空间。举个例子，将 combine6 的多累积变量模式扩展到 $k=10$ 和 $k=20$ ，其结果的比较如下表所示：

函数	方法	整数		浮点数	
		+	*	+	*
combine6	10×10 循环展开	0.55	1.00	1.01	0.52
	20×20 循环展开	0.83	1.03	1.02	0.68
吞吐量界限		0.50	1.00	1.00	0.50

我们可以看到对这种循环展开程度的增加没有改善 CPE，有些甚至还变差了。现代 x86-64 处理器有 16 个寄存器，并可以使用 16 个 YMM 寄存器来保存浮点数。一旦循环变量的数量超过了可用寄存器的数量，程序就必须在栈上分配一些变量。

例如，下面的代码片段展示了在 10×10 循环展开的内循环中，累积变量 acc0 是如何更新的：

```
Updating of accumulator acc0 in 10 x 10 urolling
vmulsd  (%rdx), %xmm0, %xmm0    acc0 *= data[i]
```

我们看到该累积变量被保存在寄存器 %xmm0 中，因此程序可以简单地从内存中读取 data[i]，并与这个寄存器相乘。

与之相比，20×20 循环展开的相应部分非常不同：

```
Updating of accumulator acc0 in 20 x 20 unrolling
vmovsd  40(%rsp), %xmm0
vmulsd  (%rdx), %xmm0, %xmm0
vmovsd  %xmm0, 40(%rsp)
```

累积变量保存为栈上的一个局部变量，其位置距离栈指针偏移量为 40。程序必须从内存中读取两个数值：累积变量的值和 data[i] 的值，将两者相乘后，将结果保存回内存。

一旦编译器必须要诉诸寄存器溢出，那么维护多个累积变量的优势就很可能消失。幸运的是，x86-64 有足够多的寄存器，大多数循环在出现寄存器溢出之前就将达到吞吐量限制。

5.11.2 分支预测和预测错误处罚

在 3.6.6 节中通过实验证明, 当分支预测逻辑不能正确预测一个分支是否要跳转的时候, 条件分支可能会招致很大的预测错误处罚。既然我们已经学习到了一些关于处理器是如何工作的知识, 就能理解这样的处罚是从哪里产生出来的了。

现代处理器的工作远超前于当前正在执行的指令, 从内存读新指令, 译码指令, 以确定在什么操作数上执行什么操作。只要指令遵循的是一种简单的顺序, 那么这种指令流水线化(instruction pipelining)就能很好地工作。当遇到分支的时候, 处理器必须猜测分支该往哪个方向走。对于条件转移的情况, 这意味着要预测是否会选择分支。对于像间接跳转(跳转到由一个跳转表条目指定的地址)或过程返回这样的指令, 这意味着要预测目标地址。在这里, 我们主要讨论条件分支。

在一个使用投机执行(speculative execution)的处理器中, 处理器会开始执行预测的分支目标处的指令。它会避免修改任何实际的寄存器或内存位置, 直到确定了实际的结果。如果预测正确, 那么处理器就会“提交”投机执行的指令的结果, 把它们存储到寄存器或内存。如果预测错误, 处理器必须丢弃掉所有投机执行的结果, 在正确的位置, 重新开始取指令的过程。这样做会引起预测错误处罚, 因为在产生有用的结果之前, 必须重新填充指令流水线。

在 3.6.6 节中我们看到, 最近的 x86 处理器(包含所有可以执行 x86-64 程序的处理器)有条件传送指令。在编译条件语句和表达式的时候, GCC 能产生使用这些指令的代码, 而不是更传统的基于控制的条件转移的实现。翻译成条件传送的基本思想是计算出一个条件表达式或语句两个方向上的值, 然后用条件传送选择期望的值。在 4.5.7 节中我们看到, 条件传送指令可以被实现为普通指令流水线化处理的一部分。没有必要猜测条件是否满足, 因此猜测错误也没有处罚。

那么一个 C 语言程序员怎么能够保证分支预测处罚不会阻碍程序的效率呢? 对于参考机来说, 预测错误处罚是 19 个时钟周期, 赌注很高。对于这个问题没有简单的答案, 但是下面的通用原则是可用的。

1. 不要过分关心可预测的分支

我们已经看到错误的分支预测的影响可能非常大, 但是这并不意味着所有的程序分支都会减缓程序的执行。实际上, 现代处理器中的分支预测逻辑非常善于辨别不同的分支指令的有规律的模式和长期的趋势。例如, 在合并函数中结束循环的分支通常会被预测为选择分支, 因此只在最后一次会导致预测错误处罚。

再来看另一个例子, 当从 combine2 变化到 combine3 时, 我们把函数 get_vec_element 从函数的内循环中拿了出来, 考虑一下我们观察到的结果, 如下所示:

函数	方法	整数		浮点数	
		+	*	+	*
combine2	移动 vec_length	7.02	9.03	9.02	11.03
combine3	直接数据访问	7.17	9.02	9.02	11.03

CPE 基本上没变, 即使这个转变消除了每次迭代中用于检查向量索引是否在界限内的两个条件语句。对这个函数来说, 这些检测总是确定索引是在界内的, 所以是高度可预测的。

作为一种测试边界检查对性能影响的方法, 考虑下面的合并代码, 修改 combine4 的

内循环，用执行 `get_vec_element` 代码的内联函数结果替换对数据元素的访问。我们称这个新版本为 `combine4b`。这段代码执行了边界检查，还通过向量数据结构来引用向量元素。

```
1  /* Include bounds check in loop */
2  void combine4b(vec_ptr v, data_t *dest)
3  {
4      long i;
5      long length = vec_length(v);
6      data_t acc = IDENT;
7
8      for (i = 0; i < length; i++) {
9          if (i >= 0 && i < v->len) {
10             acc = acc OP v->data[i];
11         }
12     }
13     *dest = acc;
14 }
```

然后，我们直接比较使用和不使用边界检查的函数的 CPE：

函数	方法	整数		浮点数	
		+	*	+	*
<code>combine4</code>	无边界检查	1.27	3.01	3.01	5.01
<code>combine4b</code>	有边界检查	2.02	3.01	3.01	5.01

对整数加法来说，带边界检测的版本会慢一点，但对其他三种情况来说，性能是一样的。这些情况受限于它们各自的合并操作的延迟。执行边界检测所需的额外计算可以与合并操作并行执行。处理器能够预测这些分支的结果，所以这些求值都不会对形成程序执行中关键路径的指令的取指和处理产生太大的影响。

2. 书写适合用条件传送实现的代码

分支预测只对有规律的模式可行。程序中的许多测试是完全不可预测的，依赖于数据的任意特性，例如一个数是负数还是正数。对于这些测试，分支预测逻辑会处理得很糟糕。对于本质上无法预测的情况，如果编译器能够产生使用条件数据传送而不是使用条件控制转移的代码，可以极大地提高程序的性能。这不是 C 语言程序员可以直接控制的，但是有些表达条件行为的方法能够更直接地被翻译成条件传送，而不是其他操作。

我们发现 GCC 能够为以一种更“功能性的”风格书写的代码产生条件传送，在这种风格的代码中，我们用条件操作来计算值，然后用这些值来更新程序状态，这种风格对立于一种更“命令式的”风格，这种风格中，我们用条件语句来有选择地更新程序状态。

这两种风格也没有严格的规则，我们用一个例子来说明。假设给定两个整数数组 `a` 和 `b`，对于每个位置 `i`，我们想将 `a[i]` 设置为 `a[i]` 和 `b[i]` 中较小的那一个，而将 `b[i]` 设置为两者中较大的那一个。

用命令式的风格实现这个函数是检查每个位置 `i`，如果它们的顺序与我们想要的不同，就交换两个元素：

```
1  /* Rearrange two vectors so that for each i, b[i] >= a[i] */
2  void minmax1(long a[], long b[], long n) {
3      long i;
```

```

4      for (i = 0; i < n; i++) {
5          if (a[i] > b[i]) {
6              long t = a[i];
7              a[i] = b[i];
8              b[i] = t;
9          }
10     }
11 }

```

在随机数据上测试这个函数，得到的 CPE 大约为 13.50，而对于可预测的数据，CPE 为 2.5~3.5，其预测错误惩罚约为 20 个周期。

用功能式的风格实现这个函数是计算每个位置 i 的最大值和最小值，然后将这些值分别赋给 $a[i]$ 和 $b[i]$ ：

```

1  /* Rearrange two vectors so that for each i, b[i] >= a[i] */
2  void minmax2(long a[], long b[], long n) {
3      long i;
4      for (i = 0; i < n; i++) {
5          long min = a[i] < b[i] ? a[i] : b[i];
6          long max = a[i] < b[i] ? b[i] : a[i];
7          a[i] = min;
8          b[i] = max;
9      }
10 }

```

对这个函数的测试表明无论数据是任意的，还是可预测的，CPE 都大约为 4.0。（我们还检查了产生的汇编代码，确认它确实使用了条件传送。）

在 3.6.6 节中讨论过，不是所有的条件行为都能用条件数据传送来实现，所以无可避免地在某些情况中，程序员不能避免写出会导致条件分支的代码，而对于这些条件分支，处理器用分支预测可能会处理得很糟糕。但是，正如我们讲过的，程序员方面用一点点聪明，有时就能使代码更容易被翻译成条件数据传送。这需要一些试验，写出函数的不同版本，然后检查产生的汇编代码，并测试性能。

 **练习题 5.9** 对于归并排序的合并步骤的传统的实现需要三个循环[98]：

```

1  void merge(long src1[], long src2[], long dest[], long n) {
2      long i1 = 0;
3      long i2 = 0;
4      long id = 0;
5      while (i1 < n && i2 < n) {
6          if (src1[i1] < src2[i2])
7              dest[id++] = src1[i1++];
8          else
9              dest[id++] = src2[i2++];
10     }
11     while (i1 < n)
12         dest[id++] = src1[i1++];
13     while (i2 < n)
14         dest[id++] = src2[i2++];
15 }

```

对于把变量 $i1$ 和 $i2$ 与 n 做比较导致的分支，有很好的预测性能——唯一的预测错误

发生在它们第一次变成错误时。另一方面，值 `src1[i1]` 和 `src2[i2]` 之间的比较(第 6 行)，对于通常的数据来说，都是非常难以预测的。这个比较控制一个条件分支，运行在随机数据上时，得到的 CPE 大约为 15.0(这里元素的数量为 $2n$)。

重写这段代码，使得可以用一个条件传送语句来实现第一个循环中条件语句(第 6~9 行)的功能。

5.12 理解内存性能

到目前为止我们写的所有代码，以及运行的所有测试，只访问相对比较少量的内存。例如，我们都是在长度小于 1000 个元素的向量上测试这些合并函数，数据量不会超过 8000 个字节。所有的现代处理器都包含一个或多个高速缓存(cache)存储器，以对这样少量的存储器提供快速的访问。本节会进一步研究涉及加载(从内存读到寄存器)和存储(从寄存器写到内存)操作的程序的性能，只考虑所有的数据都存放在高速缓存中的情况。在第 6 章，我们会更详细地探究高速缓存是如何工作的，它们的性能特性，以及如何编写充分利用高速缓存的代码。

如图 5-11 所示，现代处理器有专门的功能单元来执行加载和存储操作，这些单元有内部的缓冲区来保存未完成的内存操作请求集合。例如，我们的参考机有两个加载单元，每一个可以保存多达 72 个未完成的读请求。它还有一个存储单元，其存储缓冲区能保存最多 42 个写请求。每个这样的单元通常可以每个时钟周期开始一个操作。

5.12.1 加载的性能

一个包含加载操作的程序的性能既依赖于流水线的能力，也依赖于加载单元的延迟。在参考机上运行合并操作的实验中，我们看到除了使用 SIMD 操作时以外，对任何数据类型组合和合并操作来说，CPE 从没有到过 0.50 以下。一个制约示例的 CPE 的因素是，对于每个被计算的元素，所有的示例都需要从内存读一个值。对两个加载单元而言，其每个时钟周期只能启动一条加载操作，所以 CPE 不可能小于 0.50。对于每个被计算的元素必须加载 k 个值的应用，我们不可能获得低于 $k/2$ 的 CPE(例如参见家庭作业 5.15)。

到目前为止，我们在示例中还没有看到加载操作的延迟产生的影响。加载操作的地址只依赖于循环索引 i ，所以加载操作不会成为限制性能的关键路径的一部分。

要确定一台机器上加载操作的延迟，我们可以建立由一系列加载操作组成的一个计算，一条加载操作的结果决定下一条操作的地址。作为一个例子，考虑函数图 5-31 中的函数 `list_len`，它计算一个链表的长度。在这个函数的循环中，变量 `ls` 的每个后续值依赖于指针引用 `ls->next` 读出的值。测试表明函数 `list_len` 的 CPE 为 4.00，我们认为这直接表明了加载操作的延迟。要弄懂这一点，考虑循环的汇编代码：

```

    Inner loop of list_len
    ls in %rdi, len in %rax
1  .L3:                                loop:
2      addq    $1, %rax                Increment len
3      movq    (%rdi), %rdi            ls = ls->next
```

```

1  typedef struct ELE {
2      struct ELE *next;
3      long data;
4  } list_ele, *list_ptr;
5
6  long list_len(list_ptr ls) {
7      long len = 0;
8      while (ls) {
9          len++;
10         ls = ls->next;
11     }
12     return len;
13 }
```

图 5-31 链表函数。其性能受限于加载操作的延迟